

Guatemala, 17/08/2026

Universidad del Valle de Guatemala

Departamento de Ciencias de la Computación

Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Deep Learning y Sistemas Inteligentes



Proyecto 1

Competencia de Modelación

Melisa Mendizabal 23778

1. Análisis exploratorio de datos

El proyecto se desarrolló utilizando un conjunto de datos de viviendas, donde cada registro representa una propiedad y contiene información relacionada con sus características físicas, calidad, ubicación, condiciones de construcción y otros atributos que pueden influir en su precio de venta.

- **Dimensiones del dataset, tipos de variables (numéricas, categóricas, ordinales) y variable objetivo.**

La variable objetivo del problema es SalePrice, correspondiente al precio de venta de cada vivienda. El conjunto de datos inicial estaba compuesto por 1,168 registros y 81 variables. Estas variables incluían la identificación de la propiedad, características numéricas y variables categóricas relacionadas con aspectos como la zona, el tipo de vivienda, la calidad de los acabados, el garaje, el sótano, la piscina y otras condiciones de la propiedad.

Dentro de las 81 variables se identificaron 38 variables numéricas, de las cuales 3 eran continuas y 35 discretas, además de 43 variables categóricas. Algunas de estas variables categóricas representaban escalas de calidad u otras características que poseen un orden natural, por lo que se trataron posteriormente como variables ordinales.

Entre las variables numéricas se encuentran LotArea, OverallQual, YearBuilt, TotalBsmtSF, 1stFlrSF, GrLivArea, GarageCars y SalePrice, entre otras. Entre las variables categóricas se encuentran Neighborhood, MSZoning, HouseStyle, KitchenQual, GarageType, BsmtQual, HeatingQC y SaleCondition.

- **Estadísticas descriptivas (media, mediana, desviación estándar, rangos) de las variables relevantes.**

La variable objetivo presentó una media de aproximadamente \$181,441.54, una mediana de \$165,000, una desviación estándar de \$77,263.58, un valor mínimo de \$34,900 y un máximo de \$745,000. La diferencia entre la media y la mediana, junto con la distribución observada, indicó una concentración de viviendas en precios relativamente moderados y una cola hacia precios elevados.

Un resultado particularmente importante fue la asimetría de SalePrice, cuyo coeficiente de skewness fue de 1.7431. Esto mostró que la variable objetivo presentaba una distribución con sesgo positivo, debido principalmente a la presencia de algunas viviendas con precios considerablemente superiores al resto.

Además de la variable objetivo, se revisaron las estadísticas descriptivas de las variables numéricas con mayor correlación con SalePrice, GrLivArea (área habitable) y OverallQual (calidad general de la vivienda). GrLivArea presentó una

media de 1,527.40 pies cuadrados, con una mediana de 1,473.00 y una desviación estándar de 524.43, lo que indica una dispersión considerable en el tamaño de las viviendas del dataset, con casos que van desde 334 hasta 5,642 pies cuadrados. Por su parte, OverallQual, calificada en una escala de 1 a 10, mostró una media de 6.12 y una mediana de 6.00, con una desviación estándar de 1.37, lo que sugiere que la mayoría de las viviendas se concentran en niveles de calidad media-alta, sin extremos muy marcados hacia calidades bajas.

Variable	Media	Mediana	Desv. estándar	Mínimo	Máximo
SalePrice	\$181,441.54	\$165,000	\$77,263.58	\$34,900	\$745,000
GrLivArea	1,527.40	1,473.00	524.43	334.00	5,642.00
OverallQual	6.12	6.00	1.37	1.00	10.00

- **Identificación y tratamiento de valores nulos, atípicos (outliers) y posibles inconsistencias.**

Valores nulos

El análisis de valores faltantes mostró que varias columnas contenían una cantidad considerable de valores nulos. Sin embargo, durante el análisis se determinó que estos valores no necesariamente representaban información desconocida. En determinadas variables, un valor faltante significaba realmente que la vivienda no poseía esa característica. Por ejemplo, la ausencia de información en PoolQC, GarageType, FireplaceQu, Fence o algunas variables del sótano podía representar que la propiedad simplemente no tenía piscina, garaje, chimenea, cerca o sótano. Por esta razón, eliminar estos registros o sustituirlos automáticamente por una media habría introducido información incorrecta.

Se utilizaron dos estrategias principales. En las variables categóricas donde el valor nulo significaba ausencia de una característica, se utilizó la categoría None. En variables numéricas como GarageYrBlt y MasVnrArea, la ausencia se representó mediante cero. Para LotFrontage, los valores faltantes se imputaron utilizando la mediana correspondiente al Neighborhood de cada vivienda, de manera que la estimación tomara en cuenta las diferencias entre zonas. Finalmente, el único valor faltante de Electrical se reemplazó utilizando la moda de esa variable. Después de este proceso no quedaron valores nulos en el conjunto utilizado para la modelación.

Valores atípicos e inconsistencias

Para identificar valores atípicos en el precio de venta se utilizó el rango intercuartílico. Este procedimiento identificó 48 valores atípicos en SalePrice, equivalentes aproximadamente al 4.1 % del conjunto original. Sin embargo, no

todos los valores extremos debían considerarse errores, ya que una vivienda verdaderamente costosa puede constituir un dato válido.

Por ello, el tratamiento de outliers se enfocó en detectar observaciones que además de ser extremas presentaran características poco plausibles. Se identificaron dos viviendas con valores particularmente sospechosos: ambas tenían un área habitable muy grande, superior a 4,000 unidades de superficie, pero presentaban precios de venta inferiores a \$300,000. Estas observaciones fueron retiradas del conjunto. Como resultado, el dataset utilizado para el modelamiento quedó compuesto por 1,166 registros.

También se revisaron posibles inconsistencias entre variables relacionadas. Por ejemplo, se verificó que el año de remodelación no fuera anterior al año de construcción, que el año de construcción del garaje no fuera posterior al año de venta y que no existieran casos donde GarageCars indicara ausencia de garaje mientras GarageArea tuviera un valor positivo. No se encontraron inconsistencias en estas comprobaciones.

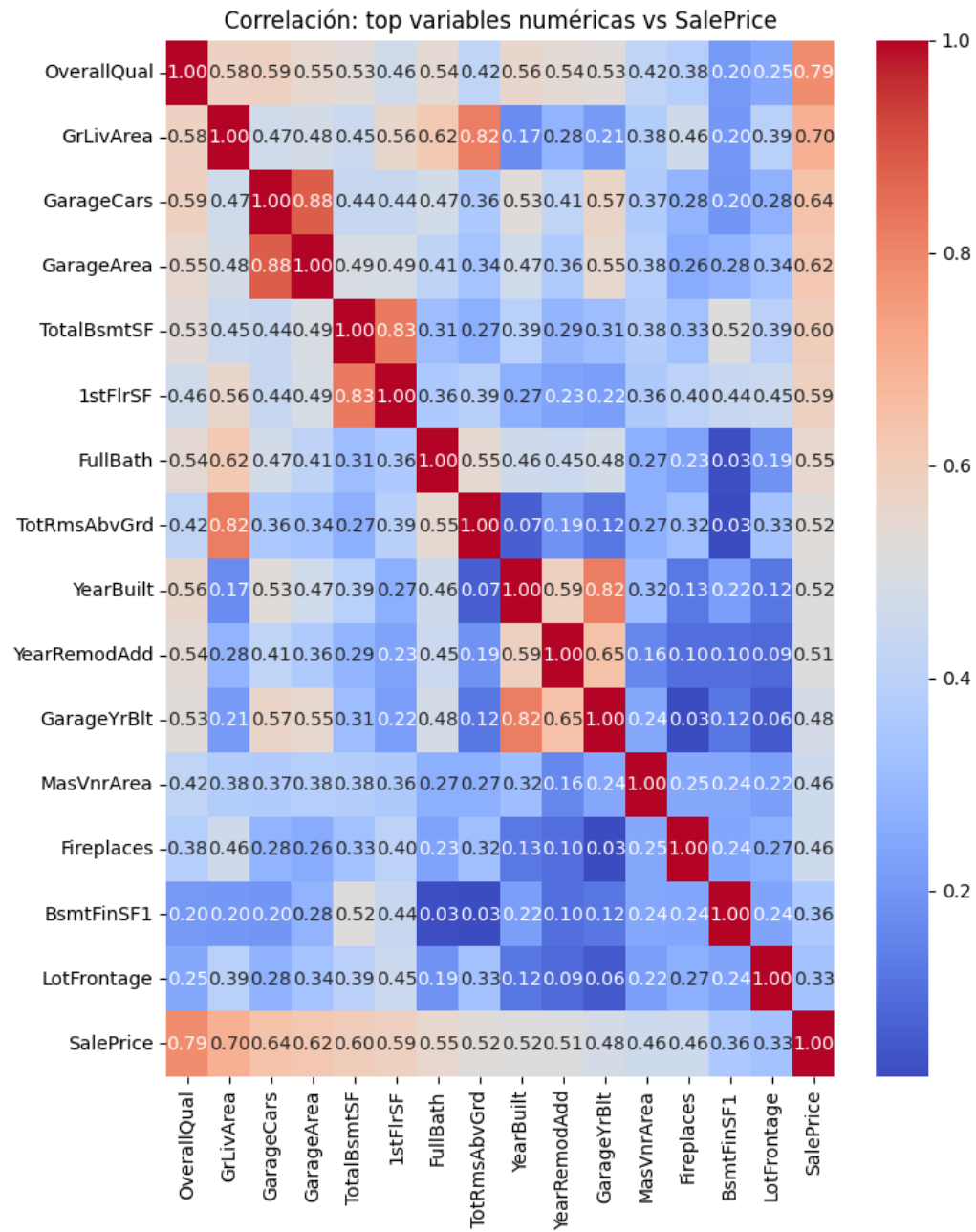
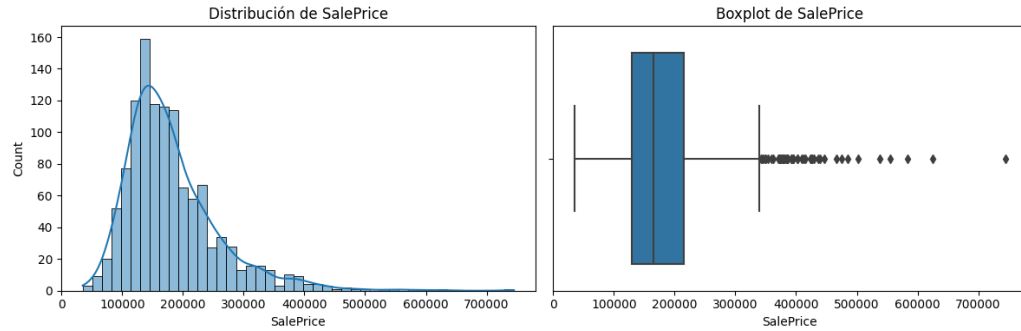
- **Visualizaciones: distribuciones de variables, correlaciones entre features, relación de cada feature con la variable objetivo.**

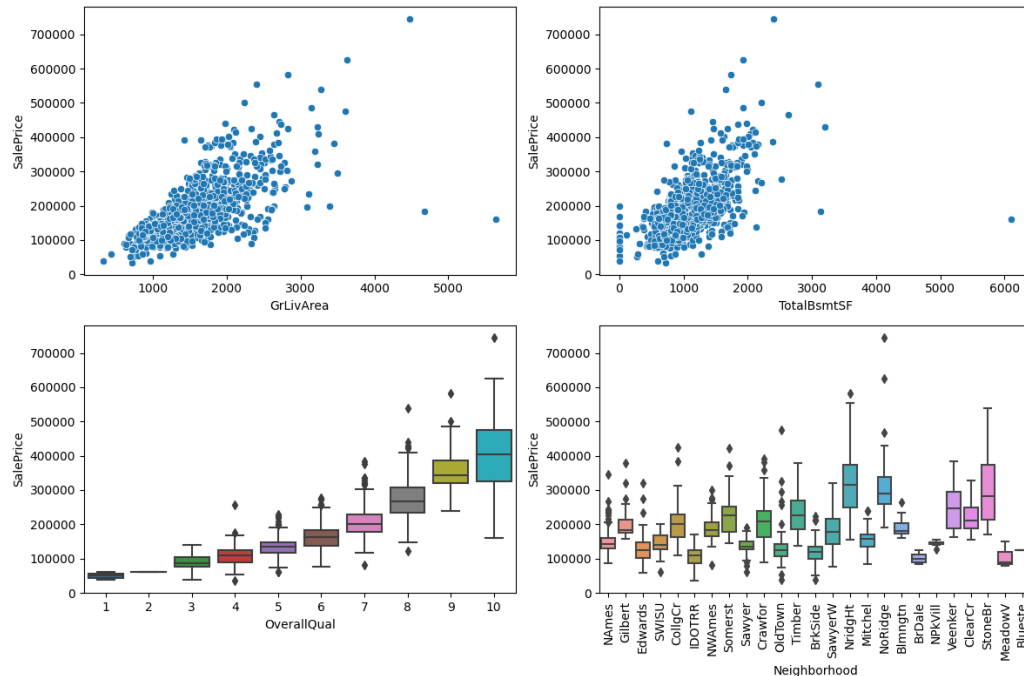
Relaciones entre las variables

El análisis de correlaciones mostró que varias variables numéricas presentaban una relación importante con el precio de venta. Entre las principales se encontraron:

Variable	Correlación con SalePrice
OverallQual	0.7856
GrLivArea	0.6957
GarageCars	0.6410
GarageArea	0.6241
TotalBsmtSF	0.5978
1stFlrSF	0.5879
FullBath	0.5525
TotRmsAbvGrd	0.5204
YearBuilt	0.5165
YearRemodAdd	0.5086

La variable con mayor correlación fue OverallQual, que representa la calidad general de la vivienda. También destacó GrLivArea, relacionada con el área habitable, lo cual es consistente con la expectativa de que viviendas de mayor tamaño y calidad tiendan a tener precios superiores.





Las visualizaciones realizadas incluyeron histogramas y diagramas de caja para SalePrice, un mapa de correlaciones y gráficos de dispersión o cajas para analizar la relación entre variables relevantes y el precio. Estos gráficos permitieron observar tanto la distribución sesgada de la variable objetivo como la relación entre características de las viviendas y su valor de venta.

- **Decisiones de preprocesamiento derivadas del EDA (normalización, codificación de categóricas, eliminación o transformación de variables) y su justificación.**

A partir del EDA se realizaron varias transformaciones antes de entrenar las redes neuronales. Las variables de calidad que presentaban un orden natural se transformaron mediante codificación ordinal. Por ejemplo, las categorías de calidad se transformaron siguiendo una escala donde None representa ausencia de la característica y los niveles progresan desde Po, Fa, TA, Gd hasta Ex.

También se utilizaron escalas específicas para variables como BsmtExposure y BsmtFinType, respetando el orden natural existente entre sus categorías.

Las variables categóricas nominales restantes se transformaron mediante one-hot encoding. Además, las categorías cuya frecuencia era inferior al 2 % fueron agrupadas en una categoría Otros, con el objetivo de reducir la dispersión provocada por categorías muy poco frecuentes.

Después del proceso de codificación se obtuvieron 151 columnas, de las cuales se retiraron Id y la variable objetivo SalePrice, dejando 149 características de entrada para los modelos.

Finalmente, las variables de entrada fueron estandarizadas mediante StandardScaler. El escalador fue ajustado únicamente utilizando el conjunto de entrenamiento y posteriormente aplicado a validación y prueba, evitando utilizar información de estos conjuntos durante el ajuste. También se aplicó una transformación logarítmica a la variable objetivo: $y_{\log} = \log_{10}(\text{SalePrice})$

Esta decisión se tomó debido al sesgo positivo observado en SalePrice. La transformación permitió reducir el efecto de los precios extremadamente altos y trabajar con una distribución más adecuada para el entrenamiento de las redes.

2. Metodología de desarrollo

El desarrollo se planteó como un proceso iterativo de búsqueda y evaluación de modelos. En lugar de seleccionar manualmente una única arquitectura, se probaron diferentes tamaños de red y combinaciones de hiperparámetros para determinar qué configuración se adaptaba mejor al conjunto de datos.

División de los datos: Los 1,166 registros resultantes del preprocesamiento se dividieron en tres conjuntos:

Conjunto	Registros	Proporción
Entrenamiento	816	70 %
Validación	175	15 %
Prueba	175	15 %

El conjunto de entrenamiento se utilizó para ajustar los pesos de las redes. El conjunto de validación se utilizó durante el desarrollo para seleccionar arquitecturas e hiperparámetros, mientras que el conjunto de prueba se reservó para la evaluación final. Esta separación permitió evitar que el conjunto de prueba influyera en la selección de los modelos.

Arquitectura MLP inicial

La arquitectura principal utilizada durante las búsquedas fue un Multilayer Perceptron (MLP) compuesto por varias capas completamente conectadas. La estructura podía variar dependiendo de la combinación evaluada.

Cada bloque de la arquitectura principal estaba compuesto por una capa Linear, seguida de BatchNorm1d, una activación ReLU y una capa Dropout. Finalmente, una capa lineal producía una única salida correspondiente al precio logarítmico estimado.

Entre las arquitecturas evaluadas se encontraban:

Estas configuraciones permitieron estudiar el efecto de aumentar tanto la profundidad como la cantidad de neuronas.

- [64, 32]
- [128, 64]
- [128, 64, 32]
- [256, 128, 64]
- [256, 128, 64, 32]

Función de pérdida y optimización

Durante la mayoría de las búsquedas se utilizó MSELoss, es decir, error cuadrático medio. Aunque el entrenamiento se realizó utilizando MSE, los resultados se reportaron mediante RMSE, ya que esta métrica permite interpretar el error en las mismas unidades de la variable objetivo.

Los modelos principales utilizaron el optimizador Adam, con tasas de aprendizaje que variaron principalmente entre 0.0005, 0.001 y 0.002. También se utilizó `weight_decay` como mecanismo de regularización L2. Entre los valores evaluados se encontraban 0.00001, 0.0001 y 0.0005. El entrenamiento utilizó lotes de 32 observaciones y se incorporó `early stopping`, deteniendo el entrenamiento cuando el desempeño de validación dejaba de mejorar durante cierto número de épocas. También se utilizó un scheduler para reducir la tasa de aprendizaje cuando el error de validación se estancaba.

Búsqueda de hiperparámetros

La primera estrategia utilizada fue una búsqueda exhaustiva mediante `grid search`. La combinación de hiperparámetros incluyó:

- arquitectura de la red
- dropout
- learning rate
- weight decay

La primera búsqueda se ejecutó tres veces con las semillas aleatorias 42, 99 y 23, con el objetivo de determinar si los resultados dependían excesivamente de la inicialización. Posteriormente se realizó una segunda búsqueda con un espacio más amplio, incorporando arquitecturas de mayor tamaño y valores de dropout más variados. Debido a que esta estrategia podía producir resultados demasiado optimistas al explorar una gran cantidad de combinaciones sobre un único conjunto de validación, se incorporó una tercera estrategia basada en validación cruzada de 5 particiones. Finalmente, se implementó una búsqueda de tipo `greedy`, en la que se ajustó un hiperparámetro a la vez, manteniendo temporalmente los demás fijos. Este procedimiento se realizó con dos espacios de búsqueda diferentes.

3. Resultados de iteraciones

El proceso de experimentación permitió observar una evolución clara del modelo. Los resultados principales de las distintas rondas se resumen a continuación.

Iteración	Método	Arquitectura ganadora	Drop out	Learning rate	Weight decay	RMSE validación log
Grid semilla 42	Grid Search	[128, 64, 32]	0.20	0.0005	0.00001	0.3544
Grid semilla 99	Grid Search	[128, 64, 32]	0.20	0.001	0.0001	0.3716
Grid semilla 23	Grid Search	[128, 64]	0.20	0.001	0.00001	0.3990
Grid ampliado	Grid Search	[256, 128, 64, 32]	0.10	0.001	0.0005	0.1962
K Fold	Validación cruzada 5 fold	[128, 64, 32]	0.15	0.001	0.00001	0.3001
Greedy 1	Búsqueda greedy	[256, 128, 64]	0.10	0.001	0.00001	0.2789
Greedy 2	Búsqueda greedy	[256, 128, 64, 32]	0.10	0.002	0.0001	0.1749

Estos resultados muestran que ampliar el tamaño de la arquitectura permitió obtener valores de RMSE de validación considerablemente menores en determinadas búsquedas. Sin embargo, estos resultados no se utilizaron directamente para seleccionar el modelo definitivo, debido a la posibilidad de sobreajustar la selección a un único conjunto de validación.

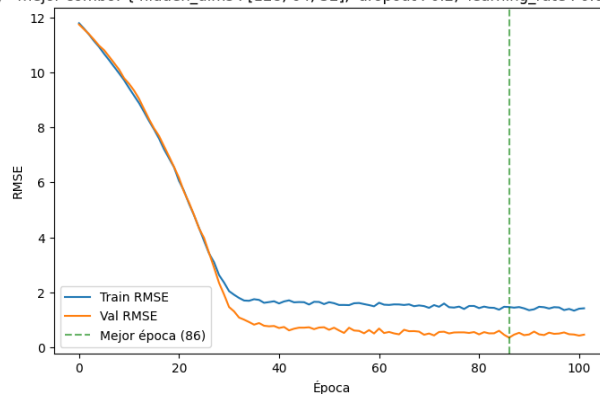
Dado que la búsqueda de hiperparámetros mediante grid y greedy search registra únicamente el RMSE de validación de cada combinación (necesario para poder comparar y seleccionar entre decenas de configuraciones sin incurrir en un costo computacional excesivo), la evidencia de overfitting se documentó a partir de las curvas de entrenamiento generadas durante las corridas individuales. En una de estas corridas, con la arquitectura [128, 64, 32] y dropout de 0.3, se observó un RMSE de entrenamiento de 2.36 frente a un RMSE de validación de 1.07 en la época 40, antes de que el early stopping detuviera el entrenamiento en la época 44.

Es importante notar que el RMSE de entrenamiento se calculó en modo train (con Dropout activo), por lo que resulta esperable que sea numéricamente mayor al de

validación, calculado en modo eval sin la pérdida de neuronas; esta diferencia no debe interpretarse como evidencia de underfitting, sino como un artefacto de cómo se mide la pérdida en cada modo. El mecanismo de early stopping, presente en todas las rondas de búsqueda, actuó como control de overfitting al detener el entrenamiento en cuanto el RMSE de validación dejaba de mejorar durante un número determinado de épocas, evitando que las arquitecturas más grandes continuaran ajustándose indefinidamente al conjunto de entrenamiento.

a. Primera ronda: búsqueda con diferentes semillas

Ronda 1 (semilla 42) - mejor combo: {'hidden_dims': [128, 64, 32], 'dropout': 0.2, 'learning_rate': 0.0005, 'weight_decay': 1e-05}

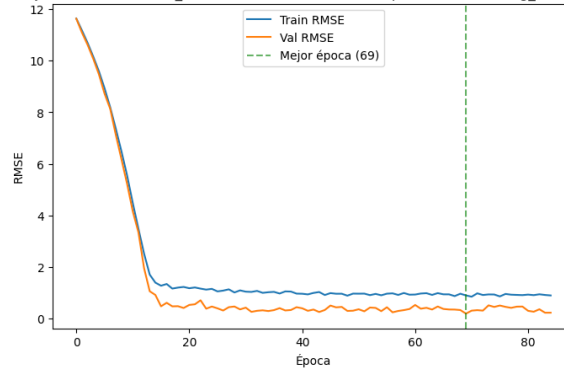


En la primera ronda se probaron 24 combinaciones utilizando tres semillas diferentes.

- Con la semilla 42, la mejor combinación fue la arquitectura [128, 64, 32], con dropout de 0.20, learning rate de 0.0005 y weight decay de 0.00001, obteniendo un RMSE de validación de 0.3544 en la escala logarítmica.
- Con la semilla 99, la mejor configuración volvió a utilizar [128, 64, 32] y dropout de 0.20, aunque el mejor learning rate fue 0.001 y el weight decay fue 0.0001, obteniendo un RMSE de 0.3716.
- Con la semilla 23, la mejor configuración fue ligeramente más pequeña, [128, 64], con dropout de 0.20, learning rate de 0.001 y weight decay de 0.00001, alcanzando un RMSE de 0.3990.
- La diferencia entre las tres semillas mostró que la inicialización aleatoria tenía un efecto apreciable sobre el resultado. Por esta razón, utilizar una única corrida no era suficiente para determinar cuál configuración generalizaba mejor.

b. Segunda ronda: grid search ampliado

Ronda 2 (grid ampliado) - mejor combo: {'hidden_dims': [256, 128, 64, 32], 'dropout': 0.1, 'learning_rate': 0.001, 'weight_decay': 0.0005}



En la segunda ronda se amplió considerablemente el espacio de búsqueda. Se evaluaron 108 combinaciones, incluyendo redes de hasta cuatro capas ocultas. La mejor configuración fue [256, 128, 64, 32], con dropout de 0.10, learning rate de 0.001 y weight decay de 0.0005, obteniendo un RMSE de validación de 0.1962.

Este resultado parecía representar una mejora importante respecto a la primera ronda. Sin embargo, el problema era que se habían evaluado muchas combinaciones sobre un conjunto de validación relativamente pequeño, compuesto solamente por 175 observaciones. Esto podía hacer que una configuración destacara parcialmente debido al azar del corte de validación.

c. Tercera ronda: validación cruzada

Para obtener una medida más confiable de generalización se realizó una búsqueda utilizando 5 fold cross validation. En esta estrategia se trabajó con 18 combinaciones y cada una se evaluó sobre cinco particiones diferentes. En cada fold se ajustó nuevamente el escalador utilizando únicamente los datos de entrenamiento de esa partición. La mejor configuración fue [128, 64, 32], con dropout de 0.15, learning rate de 0.001 y weight decay de 0.00001.

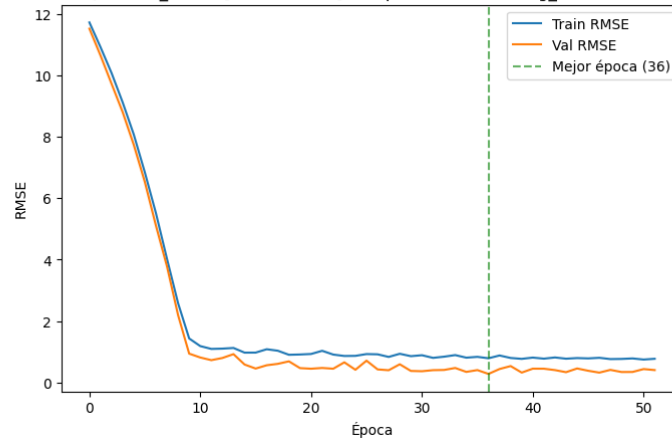
El resultado promedio fue:

- RMSE en escala logarítmica: 0.3001
- RMSE promedio en escala real: aproximadamente \$67,240

Aunque el resultado era peor que algunos de los valores obtenidos en el grid ampliado, se consideró más confiable debido a que representaba el comportamiento de la configuración sobre varias particiones del conjunto de datos. Por esta razón, esta configuración fue seleccionada inicialmente como la mejor combinación para continuar el desarrollo.

d. Búsqueda greedy

Greedy 1 - combo final: {'hidden_dims': [256, 128, 64], 'dropout': 0.1, 'learning_rate': 0.001, 'weight_decay': 1e-05}



En paralelo se experimentó con una estrategia greedy para reducir el costo computacional de explorar todas las combinaciones posibles. En el primer espacio de búsqueda, la configuración resultante fue [256, 128, 64], dropout de 0.10, learning rate de 0.001 y weight decay de 0.00001, alcanzando un RMSE de validación de 0.2789.

En el segundo espacio de búsqueda se llegó a una arquitectura todavía mayor, [256, 128, 64, 32], con dropout de 0.10, learning rate de 0.002 y weight decay de 0.0001. Esta configuración alcanzó un RMSE de validación de 0.1749.

Estos resultados reforzaron una observación importante del proyecto, aumentar la capacidad del modelo podía producir resultados muy buenos sobre validación, pero esto no garantizaba una mejor generalización sobre datos completamente nuevos.

Redes creadas y experimentos adicionales: Después de completar las búsquedas de hiperparámetros se realizaron experimentos adicionales para comprender mejor el comportamiento del modelo y determinar qué elementos de la arquitectura estaban ayudando o perjudicando el desempeño.

e. MLP con Batch Normalization

El primer modelo corresponde al MLP principal utilizado durante las búsquedas. Utiliza las capas ocultas seleccionadas mediante la validación cruzada, es decir de 149 a 128 a 64 a 32 y a 1. Cada capa oculta incorpora Batch Normalization, ReLU y Dropout. Con los hiperparámetros seleccionados mediante K Fold, el modelo obtuvo un RMSE de aproximadamente \$96,376.06 sobre el conjunto de prueba.

Aunque esta configuración había sido seleccionada inicialmente mediante validación cruzada, el desempeño sobre test mostró que la arquitectura no estaba generalizando adecuadamente.

f. MLP sin Batch Normalization

Como experimento directo se conservó la misma configuración elegida mediante K Fold, pero se eliminó Batch Normalization. La estructura mantuvo las capas lineales, ReLU y Dropout, pero ya no normalizó las activaciones entre capas. El resultado fue de RMSE test \$54,661.79

Aunque esta versión seguía presentando un error significativamente superior al de la regresión lineal, el resultado fue notablemente mejor que el modelo con BatchNorm. Esto indicó que Batch Normalization, que normalmente puede favorecer la estabilidad del entrenamiento, no estaba ayudando en este conjunto de datos particular. Este resultado también motivó una reflexión sobre la relación entre el tamaño del conjunto de datos y la complejidad del modelo.

g. MLP con camino lineal directo

El siguiente experimento buscó incorporar explícitamente la idea de conservar la estabilidad de un modelo lineal, pero sin eliminar la capacidad de una red neuronal para aprender relaciones no lineales. Para ello se implementó una arquitectura tipo Wide & Deep, compuesta por dos caminos en paralelo.

El primer camino corresponde a una transformación lineal directa de las 149 variables de entrada a una única salida. El segundo camino utiliza una pequeña red neuronal de 149, 16, 1, con una activación ReLU y Dropout de 0.20. Las salidas de ambos caminos se suman para obtener la predicción final.

Para este modelo se utilizó Adam con learning rate de 0.001, weight decay de 0.01 y la función de pérdida Huber. También se utilizó clipping de gradiente y early stopping. El modelo obtuvo un RMSE de prueba de \$88,290.87. Aunque no produjo un resultado superior a la regresión lineal, esta arquitectura fue útil para comprobar una hipótesis, una reducción importante del tamaño de la red por sí sola no garantizaba un mejor resultado.

h. MLP equivalente a sklearn MLPRegressor

También se evaluó un MLP utilizando una arquitectura equivalente a MLPRegressor con dos capas ocultas 149, 128, 64, 1. La red no utilizó Batch Normalization ni Dropout. La regularización se incorporó mediante L2. Este modelo se entrenó con un enfoque diferente al utilizado en los experimentos principales y obtuvo un RMSE sobre el conjunto de prueba de \$22,140.97.

Este resultado fue especialmente importante porque mostró que una arquitectura relativamente sencilla, acompañada de una regularización adecuada, podía competir directamente con la regresión lineal y superar ampliamente a los modelos con BatchNorm y a la arquitectura Wide & Deep.

i. MLP implementado en PyTorch con LBFGS

Finalmente se mantuvo una arquitectura equivalente a la anterior (149, 128, 64, 1), pero se sustituyó el procedimiento de optimización basado en Adam por LBFGS, utilizando todos los datos de entrenamiento simultáneamente.

Este cambio fue relevante porque el conjunto de entrenamiento es pequeño, con únicamente 816 observaciones, mientras que el modelo trabaja con 149 características de entrada. En este contexto, un optimizador basado en full batch podía resultar más adecuado que el entrenamiento tradicional mediante pequeños lotes. Se realizó además un pequeño barrido sobre el nivel de regularización L2, utilizando valores de 0.01, 0.1, 1.0, 3.0 y 7.0.

Los resultados fueron:

L2	RMSE validación log	RMSE test
0.01	0.2575	\$51,224.15
0.10	0.1240	\$23,633.53
1.00	0.1217	\$24,426.70
3.00	0.1134	\$22,913.93
7.00	0.1087	\$21,821.31

El mejor resultado se obtuvo con $L2 = 7.0$, alcanzando un RMSE de \$21,821.31. Esta configuración se convirtió en el modelo ganador y fue exportada para la etapa final del proyecto.

4. Discusión de resultados

Los resultados obtenidos muestran una relación muy clara entre la complejidad del modelo, el tamaño del conjunto de datos y la capacidad de generalización. En un inicio, el proyecto siguió una estrategia convencional de Deep Learning, probar diferentes arquitecturas, aumentar el número de neuronas, utilizar Dropout, Batch Normalization, weight decay y ajustar el learning rate. Esta estrategia produjo algunas configuraciones con errores de validación muy bajos. Sin embargo, cuando se utilizó una metodología de evaluación más robusta y posteriormente se evaluaron los modelos sobre el conjunto de prueba, se observó que las configuraciones más grandes no necesariamente eran las que mejor generalizaban.

Una de las principales razones identificadas fue el tamaño reducido del conjunto de datos. Después del preprocesamiento se disponía solamente de 1,166 viviendas, de las cuales 816 fueron utilizadas para entrenamiento. Al mismo tiempo, la codificación de las variables categóricas incrementó el número de características a 149. Esto genera un escenario de pocas observaciones en relación con la cantidad de variables disponibles. Una red neuronal con muchas capas y cientos de neuronas tiene una capacidad de representación considerable, pero en este contexto también tiene suficiente libertad para ajustarse demasiado a los ejemplos de entrenamiento. Por esta razón, el modelo más complejo no necesariamente representó la mejor solución.

Antes de interpretar los resultados de las redes neuronales se construyó una regresión lineal utilizando exactamente los mismos datos preprocesados. El modelo alcanzó un RMSE de \$22,163.69. Este resultado fue inesperado, ya que el objetivo inicial era que una red neuronal capturara relaciones más complejas y superara claramente a un modelo lineal.

La comparación permitió identificar un punto importante, la complejidad adicional de una red neuronal solamente es beneficiosa cuando existe suficiente información para aprender esas relaciones adicionales. En este proyecto, el número reducido de observaciones y la gran cantidad de características generadas mediante one-hot encoding hicieron que algunas redes tuvieran una capacidad excesiva para el tamaño del problema. La regresión lineal, por tener una estructura mucho más restringida, presentó un comportamiento más estable y logró generalizar mejor que varias de las primeras redes.

El modelo con BatchNorm obtuvo aproximadamente \$96,376.06 de RMSE, mientras que el modelo equivalente sin BatchNorm obtuvo aproximadamente \$54,661.79. La reducción del error al eliminar BatchNorm mostró que una técnica que normalmente se utiliza para estabilizar el entrenamiento no necesariamente mejora el rendimiento en todos los conjuntos de datos. En este caso, el tamaño reducido del dataset y el tipo de arquitectura probablemente hicieron que la normalización por lotes no representara una ventaja suficiente para compensar sus efectos sobre el modelo. El resultado fue utilizado como evidencia para continuar explorando arquitecturas más simples y estrategias de regularización diferentes.

Complejidad contra generalización

Los experimentos realizados permiten observar un comportamiento consistente, las redes grandes podían alcanzar errores muy bajos durante determinadas búsquedas sobre validación, pero estos resultados no siempre se mantenían sobre datos de prueba. La búsqueda ampliada obtuvo un RMSE de validación de 0.1962 y la búsqueda greedy llegó hasta 0.1749. Sin embargo, estas cifras no se utilizaron como prueba definitiva de superioridad porque provenían de muchas combinaciones evaluadas sobre conjuntos relativamente pequeños.

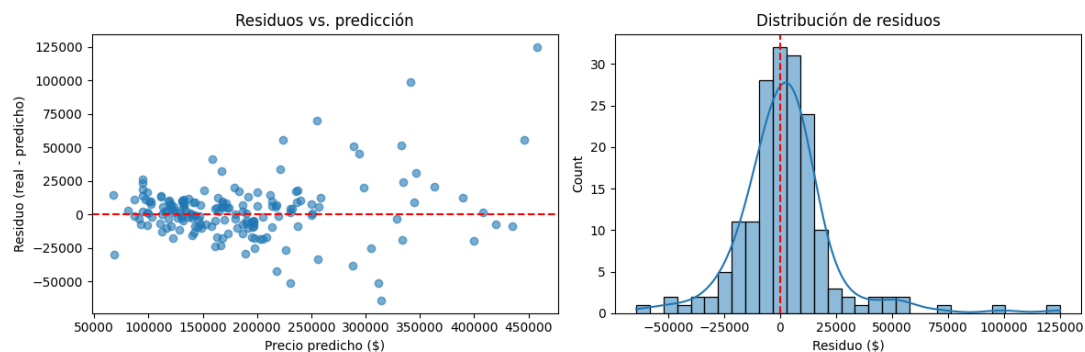
La validación cruzada permitió obtener una estimación más conservadora, seleccionando la configuración [128, 64, 32], con dropout de 0.15 y weight decay de 0.00001. Este procedimiento fue importante porque mostró que la evaluación del modelo no debía depender únicamente del menor error encontrado durante una búsqueda extensa.

El cambio más importante del proyecto ocurrió en la última etapa. Después de observar que tanto la regresión lineal como arquitecturas relativamente simples producían resultados competitivos, se planteó que el problema podía estar relacionado no solamente con la arquitectura, sino también con la forma utilizada para optimizarla. Hasta ese momento, el entrenamiento principal se había realizado utilizando Adam y lotes pequeños de datos. Posteriormente se utilizó LBFGS con

entrenamiento full batch y regularización L2. Manteniendo una arquitectura relativamente sencilla de dos capas ocultas, el cambio de optimizador produjo una mejora considerable. Con L2 igual a 7.0 se alcanzó un RMSE de \$21,821.31, convirtiéndose en el mejor resultado del proyecto. Este valor representa una mejora de aproximadamente 1.5 % respecto a la regresión lineal, cuyo RMSE fue de \$22,163.69. Aunque la mejora porcentual no es muy grande, es importante destacar que el MLP logró superar al modelo lineal utilizando una arquitectura no lineal y manteniendo un error de predicción ligeramente menor.

Análisis de residuos del modelo final

- **Residuo medio: 2612.272991071426**
- **Residuo std: 21726.548233449317**



Para evaluar si el modelo final presenta sesgos sistemáticos, se calcularon los residuos sobre el conjunto de prueba como la diferencia entre el precio real y el precio predicho. El residuo promedio fue de \$2,612, con una desviación estándar de \$21,726, lo que indica que en promedio el modelo no sobre ni subestima de forma generalizada, pero sí presenta una dispersión considerable en casos individuales.

Sin embargo, al segmentar el error por rango de precio se observó un patrón claro: el modelo no falla de manera homogénea, sino que su error crece conforme aumenta el precio de la vivienda. En el segmento de precios bajos (menores a \$150,000) el RMSE fue de \$10,621, con un residuo promedio cercano a cero (\$663). En el segmento medio (\$150,000–\$250,000) el RMSE aumentó a \$17,826, con un residuo promedio de -\$3,430, indicando una ligera tendencia a sobreestimar. En el segmento alto (mayor a \$250,000) el error se disparó a un RMSE de \$41,964, con un residuo promedio de +\$21,447, lo que muestra que el modelo subestima sistemáticamente el precio de las viviendas más caras.

Este patrón es consistente con lo observado en el gráfico de dispersión de residuos: la nube de puntos muestra una forma de embudo, con residuos cercanos a cero y poca dispersión para precios predichos bajos, y una dispersión creciente conforme el precio predicho aumenta. Los cinco errores más grandes del conjunto de prueba corresponden todos a viviendas de precio real superior a \$250,000 (entre \$250,000 y

\$582,933), con residuos de hasta \$125,128 en el caso más extremo. Esto sugiere que el modelo tiene dificultad para capturar los factores que determinan el precio en el segmento alto del mercado, probablemente porque estas viviendas son minoría dentro del conjunto de entrenamiento y presentan mayor variabilidad en características que no están bien representadas en el resto de los datos (acabados de lujo, ubicaciones particulares, remodelaciones específicas). Este hallazgo es consistente con la limitación señalada anteriormente respecto al tamaño reducido del dataset, al haber relativamente pocas observaciones de viviendas costosas, el modelo tiene menos evidencia para aprender los patrones que determinan su precio con la misma precisión que en el resto del mercado

Limitaciones

- La principal limitación del proyecto es el tamaño del conjunto de datos. La cantidad de variables después del preprocesamiento es relativamente alta respecto al número de observaciones disponibles. Aunque inicialmente contiene 1,168 registros, el proceso de codificación produce 149 variables de entrada y deja únicamente 816 observaciones destinadas al entrenamiento. Esta proporción limita la capacidad de las redes profundas para aprender representaciones complejas sin riesgo de sobreajuste.
- Otra limitación es que el modelo trabaja con un conjunto de viviendas específico, por lo que sus resultados dependen de las características y distribución presentes en dicho dataset. Un modelo que funcione correctamente para estas propiedades podría no presentar el mismo desempeño sobre viviendas pertenecientes a otra región, época o mercado inmobiliario.
- También debe considerarse que el uso de log1p sobre la variable objetivo mejora el comportamiento estadístico del problema, pero implica que las predicciones deben transformarse nuevamente a la escala original mediante $\exp(m)$ para poder interpretar el error en dólares.

Comparación final de los modelos

Modelo	Características principales	RMSE test
Regresión lineal	Modelo de referencia	\$22,163.69
MLP con BatchNorm	128, 64, 32 + BatchNorm + Dropout	\$96,376.06
MLP sin BatchNorm	128, 64, 32 + Dropout	\$54,661.79
MLP Wide & Deep	Camino lineal + red de 16 neuronas	\$88,290.87
MLP sklearn	128, 64, sin BatchNorm/Dropout, L2	\$22,140.97
MLP PyTorch + LBFGS	128, 64, full batch, L2 = 7	\$21,821.31

La tabla evidencia que las primeras arquitecturas utilizadas durante las búsquedas no lograron superar al modelo lineal cuando se evaluaron finalmente sobre test. La arquitectura Wide & Deep tampoco logró cumplir el objetivo esperado, a pesar de estar diseñada específicamente para combinar una componente lineal con una no lineal.

En cambio, el modelo equivalente a MLPRegressor consiguió acercarse considerablemente al desempeño de la regresión lineal, y el modelo final implementado en PyTorch con LBFGS consiguió superarla. La diferencia entre el modelo final y la regresión lineal fue relativamente pequeña en términos porcentuales, pero suficiente para convertir al MLP con LBFGS en el mejor modelo del conjunto de experimentos.

5. Conclusiones

- El modelo final obtuvo un RMSE de \$21,821.31, lo que significa que, en promedio, sus predicciones se alejan aproximadamente \$21,821 del precio real de las viviendas. Este resultado superó ligeramente a la regresión lineal, cuyo RMSE fue de \$22,163.69, por lo que el modelo de Deep Learning logró una mejor generalización, aunque con una diferencia relativamente pequeña.
- Uno de los principales aprendizajes fue que una mayor complejidad no garantiza un mejor desempeño. Debido al tamaño reducido del dataset y al elevado número de características después del preprocesamiento, las redes más grandes presentaron problemas de generalización. Además, la validación cruzada permitió obtener una evaluación más confiable y mostró la importancia de no seleccionar un modelo únicamente por su resultado en un solo conjunto de validación.
- Finalmente, el uso de LBFGS junto con regularización L2 permitió obtener el mejor resultado del proyecto, mostrando que el optimizador utilizado también puede tener un impacto importante según las características del dataset. Como trabajo futuro, sería conveniente contar con un conjunto de datos más grande, probar otras técnicas de selección o reducción de características y evaluar diferentes arquitecturas y métodos de optimización para determinar si es posible mejorar el desempeño manteniendo una buena capacidad de generalización.

6. Anexos

Enlace al repositorio de GitHub:

https://github.com/melisadmendizabal/deep_Proyecto1